PRODUCT REQUIREMENT DOCUMENT

Hashnode

A Developer-First Blogging & Publishing Platform

Built on the Pure MERN Stack

MongoDB · Express.js · React.js · Node.js

Attribute	Detail
Project Type	Individual Capstone Project — Full Stack Web Development
Target Level	Beginner to Intermediate
Team Size	1 Student (Solo Build)
Duration	3 Weeks (15–18 working days)
Stack	MongoDB, Express.js, React.js, Node.js (Pure MERN, No TypeScript, No SSR)
Document Owner	Full Stack Development Program
Document Status	Approved for Student Distribution

1. Executive Summary & Learning Objectives

1.1 Project Overview

Hashnode is a full-stack publishing platform built for developers to write, format, and share technical content with a public audience. The platform is purpose-built around the workflows real developers use: writing in Markdown, embedding code snippets with syntax highlighting, organizing posts by technology tags, and browsing a public feed of articles from other developers.

Every registered user gets a personal publishing dashboard where they can draft, edit, publish, and manage their own articles. Visitors — logged in or not — can browse a public feed, open individual posts, filter by tag, and view author profiles.

This is a solo build. There is no pairing, no split-by-feature team, and no backend/frontend division of labor — the student owns the entire stack from the MongoDB schema to the final deployed React interface.

Scope note: this PRD is issued for educational purposes within the Full Stack Development program. It is not affiliated with, endorsed by, or submitted to any company. Students are building their own independent application to this specification as a learning exercise.

1.2 Core Skills Students Will Master

- Designing and implementing secure user authentication using JWT and bcryptjs, including protected routes on both client and server.
- Structuring a REST API in Express.js with proper route separation, controllers, middleware, and error handling.
- Modeling relational-style data in MongoDB using Mongoose, including references between Users, Posts, and Tags.
- Handling Markdown parsing and rendering, including fenced code blocks with syntax highlighting — a core requirement for any developer blogging tool.
- Building a multi-page React application with React Router, covering public routes, protected routes, and dynamic routes (e.g. /post/:slug).
- Managing global application state (authentication state, current user) using the React Context API.
- Consuming a REST API from React using Axios, including handling loading states, error states, and token-based requests.
- Practicing realistic full-stack project structure, environment variable management, and local-to-production deployment workflow.
- Applying UI/UX fundamentals for content-heavy applications: readable typography, responsive layout, and clean content hierarchy.

2. Functional Requirements (User Stories)

Requirements are split into Must-Have items, which form the graded MVP, and Nice-to-Have items, which are optional stretch goals for students who finish early or want to push their grade higher. Students should not attempt stretch goals until every Must-Have item is complete and tested.

2.1 Must-Have (MVP) Requirements

A. User Authentication

- As a visitor, I can register for a new account using my name, email, and password.
- As a visitor, I can log in with my email and password and receive a JWT that keeps me signed in.
- As a logged-in user, my password is never stored in plain text — it must be hashed using bcryptjs before saving to the database.
- As a logged-in user, I remain authenticated across page refreshes (token persisted in localStorage and validated on load).
- As a logged-in user, I can log out, which clears my session on the client.
- As a visitor, I cannot access protected pages (Dashboard, New Post, Edit Post) without a valid token — I am redirected to the login page.

B. Post Management (CRUD)

- As a logged-in user, I can create a new post with a title, a Markdown body, a cover image URL, and one or more tags.
- As a logged-in user, my post body supports Markdown syntax, including headings, bold/italic text, links, lists, and fenced code blocks.
- As a reader, code blocks inside a published post render with language-based syntax highlighting, not as plain text.
- As a logged-in user, I can save a post as a draft or publish it immediately.
- As a logged-in user, I can edit any post I authored, including changing its title, content, tags, or cover image.
- As a logged-in user, I can delete a post I authored, with a confirmation step before deletion.
- As a logged-in user, I cannot edit or delete a post authored by someone else, even by manipulating the URL directly (enforced on the backend, not just hidden in the UI).
- As a system, every post automatically records its author, creation date, and last-updated date.

C. Tags / Topics

- As a logged-in user, I can attach one or more existing tags to a post, or create a new tag while writing (e.g. javascript, mongodb, career).
- As a reader, I can click a tag on any post and see a filtered feed of every published post under that tag.
- As a reader, I can browse a dedicated Tags page listing all available tags and how many posts use each one.

D. Public Developer Feed

- As a visitor (logged in or not), I can view a public feed of all published posts, newest first.
- As a visitor, each feed card shows the post title, a short excerpt, cover image, author name, publish date, and tags.
- As a visitor, I can search the feed by keyword matched against post titles.

- As a visitor, I can click any post card to open the full post on its own page.
- As a visitor, draft posts never appear in the public feed, tag pages, or author profile — only published posts are publicly visible.

E. User Profiles

- As a visitor, I can view any user's public profile page showing their name, bio, and a list of their published posts.
- As a logged-in user, I can edit my own profile: display name, short bio, and avatar URL.
- As a logged-in user, I have a private Dashboard listing all of my posts (drafts and published) with quick actions to edit or delete.

2.2 Nice-to-Have (Stretch Goals)

- Post likes/reactions with a count displayed on the post and feed card.
- Threaded comments on individual posts.
- Reading-time estimate calculated from word count and shown on post cards.
- Follow/unfollow other authors and a personalized feed of followed authors.
- Pagination or infinite scroll on the public feed instead of loading all posts at once.
- Image upload to a service like Cloudinary instead of pasting a cover image URL.
- Dark mode toggle persisted per user.
- Bookmark/save-for-later list per user.

3. Technical Architecture & Data Flow

3.1 High-Level Architecture

Hashnode follows a standard decoupled MERN architecture. The React client never talks to MongoDB directly — every read or write goes through the Express REST API, which is the single source of truth for validation, authorization, and data access via Mongoose.

```
[ React Client (Vite) ]
        |  Axios (HTTP/JSON, JWT in Authorization header)
        v
[ Express.js REST API ]
        |  Mongoose ODM
        v
[ MongoDB Database ]

Auth flow: Client -> POST /api/auth/login -> Server verifies
credentials -> signs JWT -> Client stores token -> Client sends
token on every protected request -> Server middleware verifies
token before reaching the controller.
```

The client is a single-page application. All routing (public feed, post detail, login, dashboard, editor) happens client-side through React Router; the server only ever returns JSON, never rendered HTML pages.

3.2 Recommended Folder Structure — client/

```
client/
├── public/
├── src/
│   ├── api/
│   │   └── axios.js          # pre-configured Axios instance
│   ├── assets/
│   ├── components/
│   │   ├── layout/           # Navbar, Footer, ProtectedRoute
│   │   ├── post/             # PostCard, PostList, TagPill
│   │   └── editor/           # MarkdownEditor, MarkdownPreview
│   ├── context/
│   │   └── AuthContext.jsx   # global auth state (user, token)
│   ├── pages/
│   │   ├── Feed.jsx
│   │   ├── PostDetail.jsx
│   │   ├── TagPage.jsx
│   │   ├── Login.jsx
│   │   ├── Register.jsx
│   │   ├── Dashboard.jsx
│   │   ├── PostEditor.jsx    # used for both create & edit
│   │   └── Profile.jsx
│   ├── hooks/
│   │   └── useAuth.js
│   ├── App.jsx
│   └── main.jsx
├── .env
└── package.json
```

3.3 Recommended Folder Structure — server/

```
server/
├── config/
│   └── db.js                # MongoDB connection
├── models/
│   ├── User.js
│   ├── Post.js
│   └── Tag.js
├── controllers/
│   ├── authController.js
│   ├── postController.js
│   ├── tagController.js
│   └── userController.js
├── routes/
│   ├── authRoutes.js
│   ├── postRoutes.js
│   ├── tagRoutes.js
│   └── userRoutes.js
├── middleware/
│   ├── authMiddleware.js    # JWT verification
│   └── errorMiddleware.js
├── utils/
│   └── generateToken.js
├── .env
├── server.js
└── package.json
```

4. Database Schema Design (Mongoose Models)

Three core collections are required for the MVP: User, Post, and Tag. Posts reference their author by User ObjectId and reference their tags by Tag ObjectId, which is how the platform relates authors, articles, and topics.

4.1 User Model

```
// models/User.js
const userSchema = new mongoose.Schema({
  name:      { type: String, required: true, trim: true },
  email:     { type: String, required: true, unique: true, lowercase: true },
  password:  { type: String, required: true },   // stored as bcrypt hash
  bio:       { type: String, maxlength: 200, default: '' },
  avatarUrl: { type: String, default: '' },
}, { timestamps: true });
```

4.2 Post Model

```
// models/Post.js
const postSchema = new mongoose.Schema({
  title:      { type: String, required: true, trim: true },
  slug:       { type: String, required: true, unique: true },
  content:    { type: String, required: true },   // raw Markdown
  excerpt:    { type: String, maxlength: 250 },
  coverImage: { type: String, default: '' },
  status:     { type: String, enum: ['draft', 'published'], default: 'draft' },
  author:     { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  tags:       [{ type: mongoose.Schema.Types.ObjectId, ref: 'Tag' }],
}, { timestamps: true });
```

4.3 Tag Model

```
// models/Tag.js
const tagSchema = new mongoose.Schema({
  name: { type: String, required: true, unique: true, lowercase: true, trim: true },
  slug: { type: String, required: true, unique: true },
}, { timestamps: true });
```

4.4 Relationship Summary

- One User can author many Posts (1 : many, via Post.author).
- One Post can have many Tags, and one Tag can belong to many Posts (many : many, via Post.tags).
- Post.slug is used for clean, readable URLs such as /post/building-a-rest-api-with-express.

5. API Specification (RESTful Endpoints)

All protected routes require a valid JWT sent as Authorization: Bearer <token>. Ownership checks (e.g. only the author can edit or delete a post) must be enforced in the controller, not assumed from the frontend.

5.1 Auth Routes — /api/auth

Method	Route	Access	Request Body	Purpose
POST	/api/auth/register	Public	{ name, email, password }	Create a new user account
POST	/api/auth/login	Public	{ email, password }	Authenticate user and return a JWT
GET	/api/auth/me	Protected	—	Return the currently logged-in user's profile

5.2 Post Routes — /api/posts

Method	Route	Access	Request Body	Purpose
GET	/api/posts	Public	— (query: search, tag)	List published posts for the public feed
GET	/api/posts/:slug	Public	—	Get a single published post by slug
POST	/api/posts	Protected	{ title, content, tags, coverImage, status }	Create a new post (draft or published)
PUT	/api/posts/:id	Protected (owner)	{ title, content, tags, coverImage, status }	Update an existing post
DELETE	/api/posts/:id	Protected (owner)	—	Delete a post
GET	/api/posts/mine	Protected	—	List all posts (draft + published) for the logged-in user

5.3 Tag Routes — /api/tags

Method	Route	Access	Request Body	Purpose
GET	/api/tags	Public	—	List all tags with post counts
GET	/api/tags/:slug/posts	Public	—	List published posts under a specific tag

5.4 User Routes — /api/users

Method	Route	Access	Request Body	Purpose
GET	/api/users/:id	Public	—	Get a public profile and their published posts

Method	Route	Access	Request Body	Purpose
PUT	/api/users/me	Protected	{ name, bio, avatarUrl }	Update the logged-in user's own profile

6. Frontend Components & Page Breakdown

6.1 Pages (React Router routes)

Route	Page Component	Access	Purpose
/	Feed.jsx	Public	Public feed of all published posts with search
/post/:slug	PostDetail.jsx	Public	Full rendered post with Markdown + syntax highlighting
/tag/:slug	TagPage.jsx	Public	Feed filtered to a single tag
/login	Login.jsx	Public	Login form
/register	Register.jsx	Public	Registration form
/dashboard	Dashboard.jsx	Protected	Logged-in user's list of drafts and published posts
/editor/new	PostEditor.jsx	Protected	Create a new post
/editor/:id	PostEditor.jsx	Protected (owner)	Edit an existing post
/profile/:id	Profile.jsx	Public	Public author profile and their posts
/settings	ProfileSettings.jsx	Protected	Edit the logged-in user's own profile

6.2 Key Reusable Components

- Navbar — shows Login/Register when logged out, Dashboard/Avatar/Logout when logged in.
- ProtectedRoute — wraps any route that requires authentication and redirects to /login if no valid token is present.
- PostCard — feed list item: cover image, title, excerpt, author, date, tag pills.
- PostList — renders a collection of PostCard components, used on Feed, TagPage, and Profile.
- TagPill — small clickable tag badge linking to /tag/:slug.
- MarkdownEditor — textarea for Markdown input paired with a live MarkdownPreview panel.
- MarkdownPreview — renders Markdown to HTML and applies syntax highlighting to fenced code blocks.
- AuthForm — shared form layout used by both Login and Register pages.
- LoadingSpinner and ErrorMessage — shared feedback components used across all data-fetching pages.

6.3 State Management Expectations

- Global state (React Context API): current authenticated user, JWT token, and auth status (loading / authenticated / guest) — managed in AuthContext and exposed via a useAuth hook.
- Local state (useState / useReducer): all form inputs, per-page fetched data (feed posts, single post, profile data), loading flags, and error messages stay local to the page or component that owns them.
- Students are not required to use Redux or any external state library — Context API plus local state is sufficient and is the expected approach for this project.

7. Suggested Step-by-Step Implementation Phases

The project is scoped for one person working roughly 4–6 hours per day across 3 weeks (15–18 working days). Each phase ends with a working, testable checkpoint — do not move to the next phase until the current one runs end-to-end.

Phase 1 — Backend Foundation (Days 1–4)

- Initialize the Express server, connect to MongoDB with Mongoose, and set up environment variables.
- Build the User, Post, and Tag models.
- Implement registration and login with bcryptjs hashing and JWT signing.
- Build the authMiddleware for verifying tokens on protected routes.
- Checkpoint: register and log in successfully via Postman/Thunder Client, and confirm a protected route rejects requests without a valid token.

Phase 2 — Core Post API (Days 5–7)

- Build full CRUD endpoints for posts, including ownership checks on update/delete.
- Implement slug generation from post titles.
- Build the Tag endpoints and the tag-to-post linking logic.
- Checkpoint: create, read, update, and delete a post end-to-end through the API, and confirm draft posts are excluded from the public feed endpoint.

Phase 3 — React Foundation & Public Feed (Days 8–10)

- Scaffold the React app with Vite, set up React Router, and configure the Axios instance.
- Build AuthContext and connect Login/Register pages to the auth API.
- Build the Feed page, PostCard component, and PostDetail page with Markdown rendering and code syntax highlighting.
- Checkpoint: a logged-out visitor can browse the public feed and open a full post with correctly rendered Markdown and highlighted code.

Phase 4 — Dashboard, Editor & Profiles (Days 11–14)

- Build ProtectedRoute and wire it around Dashboard, PostEditor, and Settings.
- Build the PostEditor page (Markdown input + live preview) for both creating and editing posts.
- Build the Dashboard listing the logged-in user's drafts and published posts with edit/delete actions.
- Build the public Profile page and the Settings page for editing one's own profile.
- Checkpoint: a logged-in user can go from writing a draft to publishing it, seeing it appear on the public feed, and editing their own profile.

Phase 5 — Polish, Testing & Deployment (Days 15–18)

- Add loading states, empty states, and user-facing error messages across all pages.
- Test authorization edge cases: editing another user's post via direct URL, expired tokens, invalid form submissions.

- Apply responsive styling and clean up the reading experience for posts (typography, spacing, code block contrast).
- Deploy the backend (e.g. Render/Railway) and frontend (e.g. Vercel/Netlify), and connect them using environment variables.
- Checkpoint: the deployed public link is fully functional for both a logged-out visitor and a logged-in author.

8. Instructor Evaluation Criteria / Grading Rubric

Each category is scored out of the listed points for a 100-point total. Partial credit applies within each category based on completeness and correctness.

Category	Points	What Is Evaluated
Authentication & Security	20	JWT implementation, bcrypt hashing, protected routes enforced on both client and server, no sensitive data exposed in responses
Post CRUD & Ownership	20	Full create/read/update/delete flow works; only the author can edit/delete their own posts, verified server-side
Markdown & Code Rendering	15	Markdown renders correctly; fenced code blocks show syntax highlighting; content is safely rendered
Tags & Public Feed	15	Tag filtering works correctly; feed excludes drafts; search functions as expected
API Design & Code Quality	15	RESTful route naming, clear controller/route separation, consistent error handling, readable code and commit history
UI/UX & Responsiveness	10	Clean, readable layout; usable on mobile widths; sensible loading/empty/error states
Deployment	5	Application is live and reachable at a public URL with working client-server connection

8.1 Baseline Pass Requirements

Regardless of point total, a submission cannot pass if any of the following are true:

- Passwords are stored in plain text or a working password hashing step is missing.
- Any user can edit or delete another user's post by calling the API directly (missing server-side ownership check).
- Draft posts are visible in the public feed.
- The application does not run locally from the submitted repository following the student's own README.

9. Tools Required

9.1 Development Environment

Tool	Purpose
Node.js (LTS) & npm	JavaScript runtime and package manager for both client and server
Visual Studio Code	Primary code editor
MongoDB Community Server or MongoDB Atlas	Local or cloud-hosted database
MongoDB Compass	GUI for inspecting collections and documents while debugging
Postman or Thunder Client	Manual API testing before the frontend is connected
Git & GitHub	Version control and code submission
Chrome/Firefox DevTools	Debugging network requests, console errors, and responsive layout

9.2 Core npm Packages — server/

Package	Purpose
express	Web server and REST routing
mongoose	MongoDB object modeling
jsonwebtoken	Issuing and verifying JWTs
bcryptjs	Password hashing
cors	Allow the React client to call the API from a different port/origin
dotenv	Load environment variables from .env
nodemon (dev dependency)	Auto-restart the server during development

9.3 Core npm Packages — client/

Package	Purpose
react, react-dom	Core UI library
react-router-dom	Client-side routing
axios	HTTP requests to the Express API
react-markdown	Rendering Markdown content as HTML
react-syntax-highlighter (or highlight.js)	Syntax highlighting for code blocks inside posts
vite	Development server and build tool

9.4 Deployment Platforms (Free-Tier Suitable)

- Backend hosting: Render or Railway.
- Frontend hosting: Vercel or Netlify.
- Database hosting: MongoDB Atlas (free M0 cluster).